

PyCommend VNS: A Multi-Objective Python Library Recommendation Framework

Augusto Magalhães Pinto de Mendonça¹, Filipe Pessoa Sousa², and Igor Machado Coelho¹

¹ Instituto de Computação, Universidade Federal Fluminense, Niterói, RJ, Brazil
`{augustompm, imcoelho}@id.uff.br`

² Instituto de Matemática e Estatística, Universidade do Estado do Rio de Janeiro, Rio de Janeiro, RJ, Brazil
`filipe.sousa@pos.ime.uerj.br`

Abstract. Software developers spend approximately 42% of their time on maintenance activities rather than creating new functionality, resulting in an estimated \$300 billion loss in global GDP annually. A major contributing factor is the difficulty in discovering appropriate existing libraries in fragmented ecosystems like Python's PyPI, which hosts over 641,000 packages without standardized categorization. This work presents PyCommend VNS, a multi-objective Variable Neighborhood Search framework for Python library recommendation that balances three competing objectives: maximizing linked usage (co-occurrence in real projects), maximizing semantic similarity (topical coherence), and minimizing recommendation set size (conciseness). We formulate library recommendation as a multi-objective combinatorial optimization problem and solve it using a tailored VNS approach with three specialized neighborhood structures (addition, removal, and swap operations) and Pareto-based local search. The framework was evaluated on a dataset of 10,000 top PyPI packages and 24,000 GitHub projects, with dependency extraction from requirements.txt files. Experimental results on 10 context libraries (FastAPI, scikit-learn, XGBoost, Prophet, OR-Tools, etc.) demonstrate that MOVNS achieves 5.1% higher hypervolume (0.748 vs 0.712) and 15.2% faster convergence compared to NSGA-II, while maintaining similar computational efficiency (7s vs 8s). The recommended library sets exhibit strong alignment with real-world ecosystem patterns, validating both technical compatibility and workflow integration. PyCommend VNS provides diverse Pareto-optimal solutions ranging from concise recommendations (2-3 libraries) to comprehensive toolkits (6-7 libraries), enabling developers to select packages aligned with their specific priorities and reducing time wasted on library discovery and reinventing existing solutions.

Keywords: Multi-Objective Optimization · Variable Neighborhood Search · Package Recommendation · Software Reuse · Library Discovery · Python Ecosystem · MOVNS · Pareto Optimization

1 Introduction

Modern software development relies heavily on third-party libraries, with the majority of projects incorporating dozens of external dependencies. Despite this reliance, discovering appropriate libraries remains challenging. Developers often implement solutions that already exist in mature, tested libraries, leading to duplicated effort and accumulated technical debt. The root causes include fragmented package ecosystems, limited search capabilities, and absence of standardized categorization systems.

The Python Package Index (PyPI) currently hosts over 641,000 packages, creating substantial discovery challenges. Unlike Maven Central’s 300,000+ libraries which maintain some categorization through group-id structure, PyPI lacks a standardized taxonomy. Auch et al. [1] demonstrated that even in the relatively organized Maven ecosystem, only 75% of libraries have tags, and these tags vary significantly in quality. They achieved 94.12% classification accuracy using a hybrid approach combining id-based and tag-based features, but their work focused on library classification rather than recommendation. The pip search functionality has been deprecated, no standardized category system exists, and package metadata remains limited and inconsistent, forcing developers to rely on manual search through documentation or community recommendations.

Xu et al. [3] identified three primary barriers to effective library reuse. First, knowledge barriers limit awareness of existing solutions, particularly for developers new to an ecosystem or working across multiple technology stacks. Second, psychological barriers lead developers to prefer building from scratch due to trust issues with external code or concerns about learning costs. Third, technical barriers in evaluating library quality, understanding integration complexities, and managing dependencies create friction in adoption. These barriers contribute to the phenomenon of "reinventing the wheel," where developers implement functionality that already exists in established libraries.

A developer seeking a web framework might discover FastAPI through GitHub stars or documentation, but identifying its common companion libraries (pydantic for data validation, uvicorn for ASGI serving, starlette for web utilities) requires either prior ecosystem knowledge or trial-and-error experimentation. This discovery problem becomes more acute as ecosystems grow. The lack of automated recommendation systems means developers must manually explore dependency graphs, read documentation for dozens of packages, and experiment with integration patterns.

Previous work by Ouni et al. [2] on search-based software library recommendation demonstrated that multi-objective optimization can balance multiple criteria in library selection, including functionality matching, license compatibility, and API stability. However, their approach did not incorporate real-world co-occurrence patterns from project dependency files or leverage semantic similarity between package descriptions. Our work addresses these limitations by combining both dimensions: usage patterns extracted from actual projects and semantic relationships derived from package metadata.

This paper presents PyCommend VNS, a Variable Neighborhood Search framework for multi-objective Python library recommendation. The framework addresses three competing objectives: maximizing linked usage (co-occurrence frequency in real-world projects), maximizing semantic similarity (topical coherence based on package descriptions), and minimizing recommendation set size (keeping suggestions concise and actionable). We formulate library recommendation as a combinatorial optimization problem and solve it using Multi-Objective Variable Neighborhood Search (MOVNS) with three specialized neighborhood structures. Addition operators expand the recommendation set with related libraries, removal operators eliminate weakly contributing packages, and swap operators replace libraries with functionally similar alternatives. The framework maintains a Pareto archive of non-dominated solutions, allowing developers to select recommendations aligned with their specific priorities.

The approach differs from existing work in three aspects. First, it extracts real-world co-occurrence data from 24,000 GitHub projects and combines it with semantic similarity from package descriptions, capturing both usage patterns and functional relatedness. Second, it treats recommendation as an inherently multi-objective problem rather than optimizing a single weighted score, providing diverse solutions along the Pareto front. Third, it employs VNS rather than genetic algorithms, exploiting the combinatorial structure of subset selection through systematic neighborhood exploration.

Experimental evaluation on 10 context libraries (FastAPI, scikit-learn, XGBoost, Prophet, OR-Tools, Ray, Pyomo, pandas, NumPy, TensorFlow) demonstrates that MOVNS achieves 5.1% higher hypervolume (0.748 vs 0.712) compared to NSGA-II while maintaining similar computational efficiency (7s vs 8s). The framework produces recommendations ranging from concise sets (2-3 libraries) to comprehensive toolkits (6-7 libraries), exhibiting strong alignment with real-world ecosystem patterns extracted from the GitHub dataset.

The remainder of this paper is organized as follows. Section 2 reviews related work on library recommendation, multi-objective optimization in software engineering, and Variable Neighborhood Search applications. Section 3 formalizes the multi-objective library recommendation problem and describes data collection from PyPI and GitHub repositories. Section 4 details the MOVNS algorithm, including solution representation, neighborhood structures, and Pareto local search mechanisms. Section 5 presents the experimental setup, including dataset characteristics, context libraries, algorithm parameters, and performance metrics. Section 6 analyzes convergence behavior, recommended library sets, and alignment with real-world ecosystem patterns. Section 7 concludes with key findings, limitations, and directions for future research.

2 Related Work

2.1 Library Recommendation Systems

2.2 Multi-Objective Optimization in Software Engineering

2.3 Variable Neighborhood Search

3 Problem Formulation

3.1 Data Collection

3.2 Multi-Objective Optimization Model

3.3 Mathematical Formulation

4 PyCommend VNS Framework

4.1 Solution Representation

4.2 Multi-Objective VNS Algorithm

4.3 Neighborhood Structures

N_1 : Addition Operator

N_2 : Removal Operator

N_3 : Swap Operator

4.4 Pareto Local Search

4.5 Shaking Mechanism

5 Experimental Setup

5.1 Dataset Characteristics

5.2 Context Libraries

5.3 Algorithm Parameters

5.4 Performance Metrics

Hypervolume (HV)

Spread (Δ)

ϵ -indicator

6 Results and Discussion

6.1 Convergence Analysis

6.2 Recommended Library Sets

6.3 Real-World Ecosystem Patterns

6.4 Computational Performance

7 Conclusions and Future Work

7.1 Key Findings

7.2 Limitations

7.3 Future Directions

Acknowledgments. This work was supported by Brazilian funding agencies FAPERJ, CAPES and CNPq.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Auch, M., Balluf, M., Mandl, P., Wolf, C.: Towards an automated classification of software libraries. *SN Computer Science* **5**(339) (2024). <https://doi.org/10.1007/s42979-024-02654-2>
2. Ouni, A., Kessentini, M., Inoue, K., Cinneide, M.O.: Search-based software library recommendation using multi-objective optimization. *Information and Software Technology* **83**, 55–75 (2017). <https://doi.org/10.1016/j.infsof.2016.11.007>
3. Xu, B., Ye, D., Xing, Z., Xia, X., Chen, G., Li, S.: Why reinventing the wheels? an empirical study on library reuse and re-implementation. *Empirical Software Engineering* **25**, 755–789 (2020). <https://doi.org/10.1007/s10664-019-09771-0>